

Table of Contents

Studium — Retrieval and Knowledge Substrate Specification

Subsystem 3 of 7. Version 1.0. Status: build-ready.

Written against data layer specification v1.1 and agent runtime specification v1.0. Assumes the six pending data-layer v1.2 items (V1–V3, V13, review_cards retention, budget query, and the exchange_index addition surfaced by subsystem 2’s build) will be folded in before subsystem 3’s build begins, but no table this subsystem depends on is affected by any of them. The seven ratified subsystem-2 v1.1 items (R1, R2, R4, R7, R9, cost re-baseline, Opus 5 swap) are similarly noted but do not affect the retrieval boundary.

1. Overview

This document specifies retrieval for Studium: how source material becomes citable evidence that grounds every claim an agent makes. The boundary is fixed by subsystem 2’s `retrieve_passages` contract; this document specifies the interior.

The retrieval subsystem is smaller than the two before it — the problem is bounded and well-explored in the literature — but it is architecturally consequential. Every claim in a generated lecture, every source-grounded turn in a tutorial exchange, every retrieval prompt in a review session passes through it. When it works, the learner reads accurate content grounded in real textbook material; when it fails silently, the agents generate confident prose that cites passages the learner can never find. That failure mode is uniquely damaging to the trust the product depends on, which is why the “thin grounding” detection in §13 is treated as a first-class flag rather than a soft-failure log line.

A senior engineer with the data layer v1.1 spec, the agent runtime v1.0 spec, and this document should be able to build a working retrieval subsystem end-to-end: chunk source PDFs into `source_chunks`, embed them via Voyage-3, serve hybrid vector-plus-keyword search with reranking, resolve citations back to specific passages, and detect and flag thin grounding for reviewer attention. The frontend that renders citations as hoverable passage previews is spec’d separately in subsystem 4, but the resolution endpoint and passage-metadata shape this subsystem exposes is defined here.

2. Scope and non-goals

In scope.

- The full interior of `retrieve_passages(concept_id, stance, k, query_text=None)`: hybrid search, reranking, concept-graph constraints, stance-based weighting, thin-grounding detection.

- The chunking algorithm: target size, overlap, semantic-boundary preferences, structure preservation. The algorithm itself is here; the ingestion pipeline that calls it lives in subsystem 5.
- The embedding pipeline: batch shape, provider interface, model-version-swap procedure, failure and retry semantics.
- Citation resolution: how [P1] . . . [Pn] markers emitted by agents map back to specific source_chunks rows at read time.
- The schema additions this subsystem needs beyond what data layer v1.1 provides. Two are needed; both are additive.
- Cost model for embedding and reranking calls; integration with the widened cost_ledger from data layer §6.12.
- Testing strategy across the same offline/online/paid tiers established by subsystems 1 and 2.

Explicitly out of scope.

- PDF-to-text extraction. Ingestion (subsystem 5) is responsible for producing raw text and structural metadata from source documents; retrieval consumes what ingestion produces. The chunking algorithm here operates on text-plus-structure input, not on binary PDFs.
- Concept graph authoring. concept_sources rows that link concepts to curated chunks are populated by ingestion or by the domain-expert authoring workflow, both of which live in subsystem 5. Retrieval reads these links but does not create them.
- Frontend rendering of citations. This spec defines what data is available for a hover card (source title, authors, page range, section path, quoted excerpt); the visual and interaction design lives in subsystem 4.
- Prompt-side integration. The agents that call retrieve_passages are spec'd in subsystem 2. What the Lecturer or Tutor does with retrieved passages — how it cites them, how it distinguishes canonical from supporting sources in its prose — belongs there.
- Multi-modal retrieval. All sources are text-only for MVP. Diagrams in PDFs are captured as image references in section paths but are not retrievable as first-class objects. Image retrieval is a v2 concern.
- Cross-subject retrieval. Every retrieval call is scoped to a single subject via the concept_id → subject_id chain. A learner enrolled in two subjects will not, at MVP, get passages from subject B when studying concept A. Adding cross-subject retrieval requires design work that isn't warranted for the MVP subject count of one.

3. Design principles

Grounding is not a suggestion. Every non-trivial factual claim in an agent's output is expected to cite a passage from the corpus. The retrieval subsystem's job is to make grounding cheap and reliable — cheap enough that agents always call retrieval rather than reason from priors, and reliable enough that the passages returned actually support what

the agent claims. When retrieval returns fewer than three passages, the agent's output is flagged for review; when retrieval returns zero, the agent should refuse to make factual claims.

Determinism where possible, learned scoring where necessary. Chunking, indexing, and citation numbering are deterministic: given the same source, they produce byte-identical results. Search relevance and reranking are learned functions that will improve over time; both are behind interfaces so their implementations can change without breaking downstream code. The line matters for prompt caching in the agent runtime — any determinism failure in the pieces that feed the cached prefix invalidates the cache, so those pieces stay deterministic.

Citation numbering is a contract. Passages returned from a single `retrieve_passages` call are numbered `[P1] . . [Pn]` in `chunk_id`-sorted order. This is not an aesthetic choice; it is what makes the cached prefix in the agent's system prompt byte-stable across identical retrieval calls, and it is what lets citation resolution work without carrying a per-call numbering table. The dev's subsystem 2 build already pins this with a test and the fixture ordering the test depends on; this document formalizes the contract.

The curated pointer set is the ground truth; vector search is the fallback. Every concept has a small set of curated `concept_sources` rows that a domain expert (initially you, later a reviewer) has verified as authoritative for that concept. Retrieval prefers those chunks first. Vector search expands the candidate set when the curated pointers are thin or when the query text asks about something the curator did not anticipate. The layering means the system defaults to what an expert would recommend and augments with what a similarity function suggests, in that order — not the other way around.

Thin grounding is a first-class failure. Fewer than three passages for a call, or an average retrieval score below a threshold, produces a `content_review_queue` row at severity 2 and passes a flag back to the caller. The caller (usually the Lecturer) is expected to note the thin grounding in its output or in `content_artifacts.metadata` so a reviewer can see which segments were produced under weak evidentiary conditions. This is what turns silent under-grounding into a reported metric, the same shape as the dev's `unattributed_content` counter for cost attribution.

Provenance is preserved end-to-end. A passage returned by retrieval carries enough metadata to reconstruct where it came from: which source, which pages, which section of that source. The client's hover card renders this without another database round trip. When a passage is used to ground a generated artifact, the resulting `content_citations` row (data layer §6.4) preserves the same chain. There is no valid retrieval result that lacks provenance.

Cost of retrieval is bounded and measured. Each retrieval call has a small, predictable cost profile: one embedding call for the query (if constructed dynamically), one hybrid search query against Postgres (indexed, sub-100ms), one rerank call to the provider (or a self-hosted reranker when we get there), and the response assembly. All of these write to the cost tracking established by data layer §6.12 and subsystem 2 §19. No retrieval call escapes accounting.

4. Technology stack

Embeddings. Voyage-3 via the Voyage AI SDK (voyageai Python package, 0.2 or later). 1024 dimensions, matching the `vector(1024)` column type already in data layer §6.3. Alternative providers (Cohere, OpenAI) are pluggable behind the `EmbeddingProvider` protocol in §7, but not exercised in MVP.

Reranking. Voyage rerank-2 via the same Voyage SDK. Input: query + up to 100 candidates; output: reordered candidates with normalized relevance scores. Pluggable behind the `Reranker` protocol; MVP uses Voyage exclusively. Cohere Rerank 3 is the fallback if Voyage becomes unavailable.

Vector index. pgvector HNSW, already declared in data layer §6.3 with `m = 16`, `ef_construction = 64`. Query-time `ef_search` is tuned in §8. No index rebuild required for MVP corpus sizes.

Full-text index. Postgres `tsvector` via a `GENERATED ALWAYS AS` column on `source_chunks`, using english configuration for MVP. English is the only supported language in v1; multi-language corpora require per-language configurations and are a v2 concern.

Runtime. Python 3.11+ inside the same FastAPI process as the agent runtime. Retrieval is a synchronous library call from the agents' perspective — no separate service, no queue. The embedding pipeline runs as a background worker (from `ingestion_jobs`, per data layer §6.13), also in the same process at MVP scale.

Testing tiers. Same three-tier structure as subsystems 1 and 2: Tier 1 offline (algorithm correctness, structured data), Tier 2 online (real Postgres with pgvector), Tier 3 paid (real Voyage API for embedding and rerank verification).

5. Schema additions required

Two additive schema changes are needed. Both are non-breaking and both belong in the data layer v1.2 batch, alongside the six items already pending.

Addition 1: `source_chunks.chunk_type`. An enum column indicating what kind of text a chunk is. Retrieval treats different chunk types differently — headings are not returned as standalone passages, exercise chunks are preferred for lab problem generation, math chunks are preserved as units even when they push the chunk size limit.

```
CREATE TYPE chunk_kind AS ENUM (  
    'body',           -- ordinary paragraph or prose  
    'heading',        -- chapter, section, subsection title  
    'code',           -- code block; preserved intact  
    'math',           -- LaTeX display math; preserved intact  
    'figure_caption', -- caption attached to a figure or table  
    'exercise',       -- end-of-section problem or exercise  
    'reference'       -- citation, bibliography entry  
);
```

```
ALTER TABLE source_chunks
  ADD COLUMN chunk_type chunk_kind NOT NULL DEFAULT 'body';

CREATE INDEX idx_source_chunks_type
  ON source_chunks (source_id, chunk_type);
```

Default is body so existing rows migrate cleanly; the chunking algorithm (§6) assigns the correct type at ingestion time going forward.

Addition 2: `source_chunks.tsvector_text`. A generated column carrying the tsvector representation of the chunk text, for full-text search alongside vector search. Generated as STORED because reading it repeatedly is cheaper than recomputing on every query.

```
ALTER TABLE source_chunks
  ADD COLUMN tsvector_text tsvector
  GENERATED ALWAYS AS (to_tsvector('english', text)) STORED;

CREATE INDEX idx_source_chunks_tsvector
  ON source_chunks USING GIN (tsvector_text);
```

The english configuration is fixed for MVP. If a v2 subject requires French or another language, the configuration becomes a per-source or per-subject choice and this column becomes per-language.

Both additions are recorded here as data layer v1.2 candidates and should be folded into that revision alongside the six items already pending.

6. The `retrieve_passages` contract

The interface is fixed by subsystem 2 §10; this section specifies its precise semantics.

Signature.

```
from typing import Protocol
from uuid import UUID
from pydantic import BaseModel

class Passage(BaseModel):
    chunk_id: UUID
    text: str
    source_id: UUID
    source_title: str
    source_authors: list[str]
    page_start: int | None
    page_end: int | None
    section_path: list[str]
    chunk_type: str
    relevance_score: float
    retrieval_reason: str

    # chunk_kind enum value
    # 0.0 to 1.0, normalized
    # 'curated', 'vector', 'keyword',
    # 'expanded'
```

```

class RetrievalResult(BaseModel):
    passages: list[Passage]           # numbered [P1]..[Pn] in chunk_id-sorted
order                                # order
    query_used: str                   # what was actually searched
    thin_grounding: bool              # True if fewer than 3 passages or avg
score < 0.5
    review_queue_id: UUID | None      # populated if thin_grounding flagged a
review row

class PassageRetriever(Protocol):
    async def retrieve_passages(
        self,
        concept_id: UUID,
        stance: str,
        k: int = 6,
        query_text: str | None = None,
    ) -> RetrievalResult:
        ...

```

Parameters.

- `concept_id`: the focus concept. Determines the candidate pool (constrained to the concept's neighborhood in the graph, per §10).
- `stance`: one of formal, intuitive, applied, historical, default. Used to weight results toward chunks with matching `concept_sources.role` (per §11).
- `k`: number of passages to return. Default 6, matching the Lecturer's request in subsystem 2 §10. Maximum enforced at 20 to bound reranking cost.
- `query_text`: optional free-form query. When `None`, the query is constructed from concept metadata (title + short_description). When provided, the query is `f"{concept.title}: {query_text}"` — the concept name anchors the query in the right neighborhood, and the caller-supplied text sharpens the intent.

Return.

`RetrievalResult` carries the ranked passages plus retrieval-side metadata. `passages` is sorted by `chunk_id` (not by relevance) so the numbering `[P1]..[Pn]` the caller assigns is byte-stable — subsystem 2 §17 requires this for prompt caching. Relevance is preserved via `relevance_score` on each passage; a caller that wants the relevance-ordered view sorts on that field.

Semantics.

`retrieve_passages` never raises for lack of results. Zero results returns an empty `passages` list with `thin_grounding=True` and a `review_queue_id`. The caller is responsible for deciding what to do — the Lecturer's expected behavior (subsystem 2 §10) is to note thin grounding to the trace and refuse to make claims that require citation.

retrieve_passages may return more passages than k in exactly one case: when a chunk is part of a math or code block and returning it without its sibling chunks would produce incomplete evidence. The algorithm in §6 keeps such blocks intact, so a single logical passage may occupy two or three source_chunks rows. The k is a soft ceiling; the hard maximum is $k + 3$.

Cost profile. One embedding call for the query when query_text is provided (~\$0.00006 for Voyage-3 at 300-token queries). One Postgres hybrid search (indexed, ~30-80ms). One rerank call over up to 20 candidates (~\$0.001). Total per retrieval: roughly \$0.002 with warm caches, dominated by the rerank call. Accounted to the calling session via ingestion_jobs for embedding costs and directly to agent_traces via subsystem 2's cost path for query embedding.

7. Chunking strategy

Chunking transforms raw source text into source_chunks rows. The algorithm lives in studium.retrieval.chunking; the ingestion pipeline (subsystem 5) calls it. Keeping the algorithm here and the pipeline there means retrieval owns chunking quality (which drives retrieval quality) without owning the mechanics of extracting text from PDFs.

Target parameters.

- **Target size:** 400 tokens per chunk, measured via Voyage-3's tokenizer.
- **Maximum size:** 600 tokens. A chunk that would exceed this splits.
- **Minimum size:** 80 tokens for body chunks; smaller chunks either merge with a neighbor or attach to a heading. code, math, figure_caption, and exercise chunks are exempt from the minimum — they may be smaller.
- **Overlap:** 15% between adjacent body chunks. Overlap is one-directional: chunk N carries a suffix from chunk $N-1$, chunk $N+1$ carries a suffix from chunk N . This means every content window is visible in exactly one chunk plus the overlap of its neighbor, which is the property that prevents context loss at boundaries without doubling indexing cost.

Break-preference hierarchy. When a chunk approaches the target size, the algorithm looks for a break point in this order of preference:

1. Chapter or section boundary (from section_path structural markers).
2. Paragraph break (blank line in source).
3. Sentence boundary (period + whitespace + capital, with abbreviation exception list).
4. Word boundary (whitespace).
5. Character boundary (if a single "word" is longer than the max chunk size, cut mid-word — very rare, usually indicates a URL or extremely long identifier).

The algorithm walks forward from the current position, accepts the first preference-1 break within the target window (350-450 tokens), then falls back to preference 2 if none, and so on.

Structure preservation. Every chunk carries a `section_path` reflecting its position in the source’s structural hierarchy:

```
["Chapter 3: Types", "3.2 Simple Types", "3.2.1 Function Types"]
```

The path is populated by the ingestion pipeline from the PDF’s outline or from heading detection; the chunking algorithm inherits the current path at the start of each chunk. When a chunk crosses a structural boundary (rare, since preference 1 tries to prevent it), the `section_path` reflects the section the chunk started in.

Chunk type assignment. The algorithm assigns `chunk_type` based on content patterns:

- **heading:** the chunk is a single line matching a heading pattern (numeric prefix, all-caps, or matching a known heading style from the source’s structure metadata). Headings are stored but never returned by `retrieve_passages` as standalone results — they exist to populate `section_path` for their siblings.
- **code:** the chunk is entirely within a code block (fenced by triple backticks in Markdown source, or detected via monospace font runs in PDF source).
- **math:** the chunk is entirely within a display-math block (LaTeX `\begin{equation}` or similar, or detected via `$$...$$` markers).
- **figure_caption:** the chunk begins with “Figure N.M:” or “Table N.M:” or similar caption markers.
- **exercise:** the chunk is within an exercises section (detected via structural context: `section_path` contains “Exercises” or “Problems”).
- **reference:** the chunk is within a bibliography or references section.
- **body:** everything else — the default.

Special handling for code and math. Code and math blocks are treated atomically: the whole block is one chunk regardless of size, up to a hard limit of 2,000 tokens. Splitting a code block or a mathematical derivation mid-way produces chunks that cite as evidence for claims neither chunk actually supports. If a code or math block exceeds 2,000 tokens, it splits at the nearest natural boundary (function boundary in code, equation boundary in math), and the resulting chunks link via a `chunk_relations` mechanism deferred to v1.1 (see §19).

Determinism. Given the same input (text plus structural metadata), the algorithm produces byte-identical `source_chunks` rows. A Tier 1 test in §17 asserts this.

8. Embedding pipeline

Embeddings are computed by a background worker triggered from `ingestion_jobs` rows with `kind = 'embed'`. The worker lives in `studium.retrieval.embedding_worker` and processes jobs asynchronously.

Batch shape. Voyage-3’s maximum batch is 128 texts per API call. The worker reads up to 128 unembedded chunks (`source_chunks` rows with no corresponding `source_chunk_embeddings` row) at a time, sends them as one batch, and inserts the resulting vectors in one transaction. If any chunk in the batch fails (rare — usually a length

overrun), the whole batch is retried once with the failing chunk excluded; if failures persist, the specific `source_chunks` rows are marked with a `content_review_queue` entry at severity 3 and the worker moves on.

Model version tracking. Every `source_chunk_embeddings` row carries `model_version` = 'voyage-3.1' (or the specific version string returned by the API). When the embedding model changes, the migration path is:

1. Add a new `source_chunk_embeddings_v2` table with the new dimensionality.
2. Run a batch re-embedding job across all chunks.
3. Once complete, atomically swap: `ALTER TABLE ... RENAME`. Retrieval reads from the new table; the old is dropped after a 7-day grace window.

This is the pattern data layer §6.3 already anticipated (“If the embedding model changes, the migration path is: add a new embedding table variant, re-embed in the background, cut over reads once complete, drop the old table.”).

Retry and backoff. Voyage API failures are retried with exponential backoff (1s, 4s, 16s, 64s), capped at four attempts. Persistent failures land in `content_review_queue` at severity 3 with details of the failure. The retrieval subsystem does not treat missing embeddings as errors — chunks without embeddings are simply invisible to vector search until they get embedded, and hybrid search’s keyword component still finds them.

Cost accounting. Every batch embedding call writes to the calling `ingestion_jobs.cost_usd` and to `cost_ledger.cost_ingestion_usd` (per data layer §6.12’s attribution rules). Voyage-3 pricing at MVP is \$0.06 per 1M tokens; a typical corpus of 200,000 tokens costs ~\$0.012 to embed once. Re-embedding on model change costs the same. This is the cheapest per-token line item in the system by two orders of magnitude.

Batch scheduling. Embedding jobs are FIFO by default. Load-bearing concept material (chunks linked from `concept_sources` on a concept where `is_load_bearing` = TRUE) is prioritized to the head of the queue so that critical material is searchable first when a new source lands. This is a small but real user-perceived benefit: the first learner to touch a newly-ingested concept doesn’t wait for the whole corpus to embed.

9. Hybrid search

Hybrid search combines vector similarity and keyword relevance. The two modalities catch different failure cases: vector search finds semantically related passages a keyword search would miss (the concept is discussed in different vocabulary than the query uses); keyword search finds passages containing a specific term or notation a vector search might rank low. Fusion gives the union without the sum’s noise.

Vector search. Query embedding computed via Voyage-3, then top-20 chunks by cosine similarity via pgvector HNSW:

```
SELECT c.id, c.text, c.chunk_type, c.section_path, c.page_start, c.page_end,
       s.title, s.authors, s.id AS source_id,
       (1 - (e.embedding <=> :query_embedding))::float AS vector_score
```

```

FROM source_chunks c
JOIN source_chunk_embeddings e ON e.chunk_id = c.id
JOIN sources s ON s.id = c.source_id
WHERE s.subject_id = :subject_id
      AND s.deleted_at IS NULL
      AND s.status = 'active'
      AND c.chunk_type NOT IN ('heading', 'reference')
ORDER BY e.embedding <=> :query_embedding
LIMIT 20;

```

The `chunk_type NOT IN ('heading', 'reference')` filter drops chunks that are structurally present but not useful as evidence. HNSW parameters at query time: SET LOCAL `hns.ef_search = 40` (double the return count is the standard rule).

Keyword search. BM25-style scoring via `ts_rank_cd`:

```

SELECT c.id, c.text, c.chunk_type, c.section_path, c.page_start, c.page_end,
       s.title, s.authors, s.id AS source_id,
       ts_rank_cd(c.tsvector_text, plainto_tsquery('english',
:query_text))::float AS keyword_score
FROM source_chunks c
JOIN sources s ON s.id = c.source_id
WHERE s.subject_id = :subject_id
      AND s.deleted_at IS NULL
      AND s.status = 'active'
      AND c.chunk_type NOT IN ('heading', 'reference')
      AND c.tsvector_text @@ plainto_tsquery('english', :query_text)
ORDER BY keyword_score DESC
LIMIT 20;

```

`plainto_tsquery` is used rather than `to_tsquery` because it handles free-form user or agent-generated queries gracefully (no special-character escaping burden). The `@@` operator uses the GIN index from §5’s Addition 2.

Fusion. Reciprocal Rank Fusion, standard formula with $k=60$:

```
rrf_score(chunk) = sum(1 / (60 + rank_in_modality)) across modalities
```

A chunk that appears at rank 1 in both searches scores $1/61 + 1/61 = 0.0328$. A chunk appearing at rank 1 in one and absent in the other scores $1/61 = 0.0164$. This is why RRF is preferred over simple score averaging — it handles the “present in one modality, absent in the other” case without needing to invent a score for the missing side.

The fused list of up to 40 unique chunks (20 vector + 20 keyword, deduplicated) is passed to the reranker.

Concept-graph constraint. Before hybrid search runs, the concept-graph filter (§10) may narrow the candidate pool by pre-selecting chunks via `concept_sources`. The above queries then apply within that pre-selected pool. This is described in §10 rather than repeated here.

Query construction. When `query_text` is provided to `retrieve_passages`, the effective query is `f"{concept.title}: {query_text}"`. When `query_text` is `None`, the effective query is `f"{concept.title}. {concept.short_description}"`. Both the vector embedding and the keyword search use this effective query; consistency between the two modalities matters for the fusion score to be meaningful.

Performance. MVP corpus (single subject, ~10-20 sources, ~5,000-15,000 chunks): both queries return in under 50ms with warm caches. Classroom-tier corpus (10 subjects, ~150 sources, ~150,000 chunks): both queries return in under 150ms with warm caches. University-tier corpus (100 subjects, ~2,000 sources, ~2,000,000 chunks): vector search remains sub-100ms courtesy of HNSW; keyword search may exceed 200ms and warrants partitioning by subject at that scale, which is a v2 concern for now.

10. Concept-graph-constrained retrieval

The concept graph narrows what retrieval considers. Without narrowing, a query about “reduction” in the lambda calculus subject could return passages from a Turing machines subject that happens to be in the same corpus — semantically related, pedagogically wrong.

Neighborhood definition. For a given `concept_id`, the retrieval neighborhood is:

1. The concept itself.
2. All concepts with a prerequisite edge *from* this concept in `concept_edges` (things this concept depends on).
3. All concepts with a dependency edge *from* this concept (things this concept enables).
4. All concepts with a generalization or related edge (either direction).
5. Load-bearing concepts in the same subject (weight lower, included as fallback).

The neighborhood is computed once per `retrieve_passages` call and materialized as a set of `concept_ids` used to filter `concept_sources`.

Two-phase retrieval. Retrieval proceeds in two phases:

Phase 1: Curated pointers. The `concept_sources` table carries chunks a domain expert has marked as authoritative for each concept. Retrieval first collects all chunks pointed at by `concept_sources` rows for concepts in the neighborhood:

```
SELECT cs.chunk_ids
FROM concept_sources cs
WHERE cs.concept_id = ANY(:neighborhood_concept_ids);
```

`cs.chunk_ids` is an array (data layer §6.2), so the SQL flattens it via `UNNEST`. These chunks are added to the candidate pool with a curated-source boost applied during fusion (curated chunks get their fusion score multiplied by 1.5).

Phase 2: Vector and keyword expansion. The hybrid search from §9 runs, but restricted to chunks whose `source_id` belongs to sources that are linked (via any `concept_sources` row)

to concepts in the neighborhood. This prevents a vector query about lambda calculus from surfacing passages from an unrelated source that happens to contain the word “reduction.”

The final candidate set is the union of Phase 1 (curated) and Phase 2 (expanded), deduplicated by `chunk_id`, ranked by fused score (with curated boost applied), and passed to the reranker.

When curated pointers are thin. If Phase 1 returns fewer than three chunks, the algorithm expands the neighborhood one hop further (adds concepts two edges away rather than one) and re-runs Phase 1. This prevents a concept with sparse curation from immediately falling back to unconstrained vector search. If the expanded phase still returns fewer than three, Phase 2 is used as the primary source rather than an expansion.

Cross-concept leakage. The `retrieval_reason` field on each returned Passage records whether the passage came from a curated pointer (`retrieval_reason = 'curated'`), vector expansion (`'vector'`), keyword expansion (`'keyword'`), or the neighborhood-expansion fallback (`'expanded'`). This makes the retrieval decision auditable at read time — if a lecture generates a claim citing an `'expanded'` passage, the reviewer knows to check whether the expansion was appropriate for that claim.

11. Stance-based retrieval

The stance parameter changes what passages are preferred, not what is returned as a hard filter. The mapping between stance values and `concept_sources.role` values:

Stance	Preferred role values
formal	canonical_definition, primary_exposition
intuitive	worked_example, primary_exposition
applied	worked_example, exercise
historical	historical, primary_exposition
default	primary_exposition, canonical_definition

Weighting. Chunks whose curated role matches the stance’s preferred set get a stance boost of 1.3× on their fusion score, applied after the curated boost. Chunks not linked via any `concept_sources` row (i.e., pure vector or keyword results) get no stance boost — the stance signal is only meaningful for chunks the curator has classified.

Not a hard filter. A formal stance retrieval still returns worked examples if they score highest overall; the stance shifts probability, not membership. This matters because pedagogical stance is a preference, not a constraint, and a good worked example that grounds a formal claim is more useful than a mediocre canonical definition.

Stance and prompt caching. Because stance changes the ranking of returned passages, the fused ranking is stance-specific. Two calls with different stances against the same concept will return different passages lists. This is fine — subsystem 2 §17 already keys the Lecturer’s cached prefix on (`concept_id`, `stance`, `grounding_version`), so different stances are expected to produce different (and separately cached) prefixes.

12. Citation resolution

Agent output contains inline citation markers of the form [P1], [P2], [P3-P4]. The frontend resolves these to hoverable passage previews. The resolution mechanism is defined here; the visual rendering is defined in subsystem 4.

Marker syntax. Exactly two forms are supported:

- Single: [P3] refers to passage 3.
- Range: [P3-P5] refers to passages 3, 4, and 5 (inclusive).

Multiple markers may appear adjacent: [P1][P4] refers to passages 1 and 4. Comma-separated forms ([P1, P4]) are not supported — the agent prompts (subsystem 2 §10) instruct against them.

Passage numbering contract. Passages returned by a single `retrieve_passages` call are numbered [P1]..[Pn] in `chunk_id`-sorted order (see §3 principle “Citation numbering is a contract”). The caller (typically the Lecturer) knows the mapping between passage number and `chunk_id` from the `RetrieveResult.passages` list and can persist it if needed — but for the citation-resolution path, the mapping is reconstructed from `content_citations` rows written when the `content_artifact` was created.

Storage of citations. When an agent produces an artifact containing citation markers, the calling code (subsystem 2 orchestration) parses the markers, resolves them via the retrieval result the agent had access to, and writes one `content_citations` row per resolved marker (per data layer §6.4). The passage number appears nowhere in the schema — only the `chunk_id` is stored, because `chunk_ids` are stable and passage numbers are per-call ephemera.

Resolution at read time. When the frontend renders a stored artifact containing markers, it makes one API call per artifact (not per marker) to resolve all markers at once:

```
GET /api/artifacts/{artifact_id}/citations
```

Returns:

```
{
  "artifact_id": "...",
  "citations": [
    {
      "marker": "P1",
      "chunk_id": "...",
      "source_id": "...",
      "source_title": "An Introduction to Functional Programming Through
Lambda Calculus",
      "source_authors": ["Michaelson, Greg"],
      "page_start": 42,
      "page_end": 42,
      "section_path": ["Chapter 3", "3.2 Beta Reduction"],
      "excerpt": "The reduction of a beta-redex ( $\lambda x.M$ )N proceeds by..."
    }
  ]
}
```

```

    "excerpt_start_offset": 0,
    "excerpt_end_offset": 400
  },
  ...
]
}

```

The marker is derived at read time from the `content_citations` rows: for a given artifact, its citations are sorted by `chunk_id` and numbered `P1..Pn`, matching the original retrieval's ordering.

Rendering. The frontend replaces `[P3]` in the rendered text with a superscript link that on hover shows a card with source title, authors, page reference, and excerpt. On click, the card expands to show the full chunk text. The passage view is described in subsystem 4.

Deleted or retired sources. If a sources row is soft-deleted (`deleted_at IS NOT NULL`) or retired (`status = 'retired'`) after an artifact was created, the resolution endpoint returns the citation with `source_deleted: true` and a graceful message; the frontend renders the marker as inactive rather than a broken link. This is why data layer §6.4 uses `ON DELETE RESTRICT` on the `content_citations.source_chunk_id` foreign key — hard deletion of a cited chunk is blocked at the schema level; retirement is the correct workflow, and retention is preserved.

13. Thin-grounding detection

A retrieval result is *thinly grounded* when the evidence returned is unlikely to adequately support the claims the caller might make. Two conditions trigger the flag:

1. **Count threshold.** Fewer than 3 passages returned. Anything meaningful an agent might claim about a concept typically requires evidence from multiple sources or multiple passages within a source; below three passages, the risk of the agent extrapolating beyond its evidence is high.
2. **Score threshold.** Average `relevance_score` across returned passages below 0.5 (on the normalized 0-1 scale from the reranker). A full slate of low-relevance passages is worse than a small slate of highly-relevant ones; the score threshold catches the case where hybrid search returned many marginal matches.

Both are configurable via `studium.retrieval.THIN_GROUNDING_MIN_COUNT` and `THIN_GROUNDING_MIN_AVG_SCORE`. Defaults as above.

On trigger. When either condition fires, the retrieval result carries `thin_grounding=True` and a `review_queue_id` pointing at a newly-created `content_review_queue` row:

```

INSERT INTO content_review_queue (
  session_turn_id,    -- if the retrieval was invoked from an agent turn
  source,             -- 'system_confidence'
  reason,             -- e.g. "Retrieval returned 2 passages, avg score
0.42"

```

```
severity          -- 2
) VALUES (... ) RETURNING id;
```

The queue row references `session_turn_id` (not `artifact_id`) because the artifact may not exist yet when retrieval runs; the linkage between artifact and thin grounding is established later via `content_artifacts.metadata.thin_grounding = true`.

Caller responsibility. The calling agent (Lecturer, Tutor, Reviewer) is responsible for behaving appropriately when `thin_grounding=True`. Subsystem 2 §10 specifies the Lecturer’s expected behavior: note thin grounding to the trace, either continue with reduced confidence or refuse to make claims that require grounding. The retrieval subsystem’s job is to detect and flag; the response is upstream.

What ends up in the review queue. A reviewer working through the queue sees the query that was used, the concept, the number of passages returned, the average score, and links to the session turn and any artifact ultimately produced. The reviewer’s action is typically to add new `concept_sources` rows pointing at appropriate passages, which fixes the thin grounding for future calls. This is the feedback loop that turns thin-grounding events into concrete corpus improvement.

14. Cache warming and pre-computation

Retrieval performance is dominated by the reranker call, which is a network round trip. Two forms of pre-computation reduce that cost.

Query result caching. Retrieval results for a given (`concept_id`, `stance`, `query_text_hash`) are cached in memory for 5 minutes. A cache hit skips the entire pipeline — hybrid search, reranking, everything. The cache is per-process (not distributed) because retrieval is called by the agent runtime, which lives in the same process. At multi-node scale, a shared Redis cache replaces the in-memory one; this is deferred to Infrastructure (subsystem 7) when Studium goes multi-node.

Warm cache for load-bearing concepts. At startup, the retrieval subsystem pre-warms the cache for a small set of “hot” retrieval calls: for each load-bearing concept in each active subject, run `retrieve_passages(concept_id, 'default', 6)` and cache the result. This means the first learner to touch a load-bearing concept doesn’t wait for a cold retrieval. A background job re-warms these on a rolling 5-minute schedule.

Embedding cache. Query embeddings for common queries (concept titles, standard question templates) are cached in a Postgres table `query_embedding_cache` (added in v1.1 of this spec if warranted by observed miss rates; not in MVP).

Cost-benefit. In the MVP scenario (single subject, ~11 concepts, 3-4 load-bearing), pre-warming costs about 4 retrieval calls per 5-minute window — roughly \$0.01/hour in retrieval costs to eliminate cold-start latency for load-bearing material. At classroom scale (10 subjects), pre-warming costs about \$0.10/hour. At university scale (100 subjects), pre-warming becomes a real line item and warrants either selective warming (only concepts touched in the last 7 days) or a cheaper cache tier (skip the reranker, cache the hybrid search output).

15. Cost model

Retrieval costs come from three sources: embedding calls (query embedding and re-embedding on model change), rerank calls (per retrieval), and pre-warming (background retrieval calls).

Per-retrieval cost. With Voyage-3 embedding and Voyage rerank-2:

- Query embedding: $300 \text{ tokens} \times \$0.06/1\text{M} = \$0.000018$
- Postgres hybrid search: no incremental cost
- Reranking 20 candidates against a 300-token query: $\sim \$0.001$
- Total: $\sim \$0.001$ per retrieval call

Per-embedding cost. Voyage-3 at $\$0.06/1\text{M}$ input tokens. A single 400-token chunk costs $\$0.000024$ to embed. A subject with 10,000 chunks (roughly a semester's worth of textbook material) costs $\$0.24$ to embed initially. Re-embedding on model change costs the same amount. This is trivial at MVP scale.

Pre-warming budget. $4 \text{ concepts} \times 12 \text{ warmings/hour} \times \$0.001/\text{retrieval} = \$0.048/\text{hour} = \sim \$35/\text{month}$ per single-subject MVP deployment, at continuous operation. Realistic: warming runs only during peak learner hours (say 12 hours/day), so $\sim \$17/\text{month}$. Small but not zero; if the retrieval subsystem's cost line becomes visible in the `cost_ledger`, the warming schedule is the first thing to tune.

Attribution. Retrieval costs attribute to different `cost_ledger` columns per source:

- Query embedding for a retrieval invoked by an agent $\rightarrow$ `cost_agent_usd` (attributed to the calling session's user via the agent's normal attribution path)
- Rerank call $\rightarrow$ `cost_agent_usd` (same)
- Bulk chunk embedding during ingestion $\rightarrow$ `cost_ingestion_usd` (attributed to the uploading user via `sources.uploaded_by`)
- Pre-warming calls $\rightarrow$ attributed to the system user (per data layer §6.12's system-attribution rule as established by the dev's V3 resolution)

Budget interaction. Retrieval costs are subject to the same pre-flight budget check (subsystem 2 §19). A retrieval that would exceed the caller's daily hard cap is refused with a `BudgetExceededError`, degrading gracefully to a "system is temporarily unable to look up sources" response. In practice this is unlikely to fire (retrieval is cheap), but the code path exists and is tested.

16. Error handling and degradation

Retrieval failure modes and their responses.

Failure	Response
Voyage embedding API timeout	Retry with exponential backoff (1s, 4s), then fall back to keyword-only search. Log to trace.

Failure	Response
Voyage embedding API 5xx	Same as timeout.
Voyage embedding rate limit (429)	Retry after Retry-After. If still limited, keyword-only fallback.
Voyage rerank API timeout	Retry once with 2s backoff, then skip reranking and return the top-k of the fused hybrid-search result. Note in the trace that reranking was skipped.
Voyage rerank API 5xx	Same as timeout.
Postgres vector search error (index corruption, plan failure)	Log to trace at severity 3; fall back to keyword-only. If keyword also fails, return empty result with <code>thin_grounding=True</code> .
Postgres full-text search error	Same as vector, in reverse — fall back to vector-only.
No chunks in the concept neighborhood	Return empty result with <code>thin_grounding=True</code> and a <code>content_review_queue</code> entry at severity 2 flagging that the concept has no curated sources.
Chunk text is malformed (encoding issue, extreme length)	Log at severity 2, exclude the chunk, continue with the rest.
Budget exceeded	Raise <code>BudgetExceededError</code> per subsystem 2 §21's pattern. The caller degrades.

Degradation copy for the “temporarily unable to look up sources” case lives in `studium/copy/degradation.py` alongside the other degradation messages, per subsystem 2 §21.

Circuit breaker. If Voyage embedding or rerank fails on three consecutive calls, a circuit breaker opens for 60 seconds — retrieval calls during that window skip the failing modality entirely without incurring more failed API calls. This prevents a Voyage outage from cascading into a session-blocking chain of retries.

17. Testing strategy

Same three-tier structure as subsystems 1 and 2.

Tier 1 — Offline (no LLM calls, no database).

- **Chunking determinism.** Given the same input text-plus-structure, `chunk_text()` returns byte-identical output across multiple invocations. Verified with hash comparison. Coverage guard fails if the test finds zero test cases.
- **Chunking correctness.** Fixtures for each chunk type (body, heading, code, math, figure_caption, exercise, reference) verify the algorithm assigns the correct type. Adversarial cases: a code block that would exceed the 2000-token hard limit splits at a function boundary; a paragraph with no sentence boundaries and no whitespace falls back to character-boundary cutting.

- **Passage numbering contract.** Given a fixed set of chunks, `RetrievalResult.passages` is sorted by `chunk_id`. A test with three chunks in different orders (returned by the mocked hybrid search in different orders) verifies the same numbering results every time.
- **Citation marker parsing.** `parse_citation_markers("[P1] and [P3-P5]")` returns `[1, 3, 4, 5]`. Adversarial cases: unbalanced brackets, ranges with reversed bounds, out-of-range references, unicode in the surrounding text.
- **Fusion arithmetic.** RRF formula verified against hand-computed values for small candidate lists.
- **Stance mapping.** Every stance value maps to a defined set of preferred roles.
- **Thin-grounding thresholds.** With mocked retrieval results, verify the flag fires at 2 passages, does not fire at 3, fires at `avg_score 0.49`, does not fire at `0.51`.

Tier 2 — Online (requires Postgres + pgvector).

- **HNSW query returns results.** Insert 100 chunks with known-distance embeddings, verify vector search returns them in expected order.
- **Full-text query returns results.** Insert 100 chunks with known keyword content, verify tsvector query returns matches.
- **Fusion produces sensible ordering.** Combined vector + keyword returns higher for chunks that match both modalities than for chunks that match only one.
- **Concept-graph filter narrows.** Retrieval on concept A returns chunks from sources in A's neighborhood, not from sources exclusively linked to a distant concept.
- **Curated pointer boost.** A `concept_sources` row pointing at a specific chunk causes that chunk to rank higher than a lexically identical chunk not pointed at.
- **Thin-grounding writes to review queue.** A retrieval on a concept with two curated chunks and no matching corpus results in a `content_review_queue` insert.
- **Citation resolution endpoint.** Given an artifact with three `content_citations` rows, `GET /api/artifacts/{id}/citations` returns the passages in `chunk_id`-sorted order with correct source metadata.

Tier 3 — Paid (requires Voyage API key).

- **Real embedding round trip.** A test that embeds a small fixture and asserts the returned vector has the expected 1024 dimensions and non-degenerate values.
- **Real reranking improves ordering.** Insert 20 chunks in random order, run `rerank`, verify the top-6 by rerank score includes the ground-truth relevant chunks (measured by a hand-labelled fixture).
- **End-to-end retrieval with real API.** Full `retrieve_passages` call against a seeded fixture, with real embedding and reranking, returns non-empty results in under 5 seconds.

Fixture data. Extend the shared lambda-calculus fixture from subsystem 2's build with 20 realistic chunks per source across 4 sources (one Michaelson chapter, one Church paper, one Selinger note, one exercise sheet), with `concept_sources` links, `page_start/page_end` filled in, and `section_path` populated from real structural context. The dev's subsystem 2

build already extended the fixture with concept-to-chunk links; this spec extends it further with realistic chunk lengths and structure.

CI wiring. Same pattern as data layer and agent runtime: Tier 1 gates every push, Tier 2 gates PRs touching retrieval paths, Tier 3 runs on-demand with `STUDIUM_RUN_PAID_TESTS=1`. A separate `.github/workflows/retrieval.yml` per the dev's principle that one green check spanning multiple subsystems can't tell you which is healthy.

18. Version history

v1.0 — 17 August 2026. Initial specification. Written against data layer v1.1 (with v1.2 items pending, none affecting retrieval tables) and agent runtime v1.0 (with v1.1 items pending, none affecting the retrieval boundary). Locks the `retrieve_passages` contract, the chunking algorithm parameters, the hybrid-search fusion approach, the two-phase (curated + expansion) retrieval flow, the stance-based weighting mechanism, the citation resolution protocol, and the thin-grounding detection thresholds. Names two data layer v1.2 schema additions (`chunk_type` enum and `tsvector_text` generated column).

Anticipated v1.1 candidates (not yet applied):

- If chunking against real corpora produces chunks that don't respect the size targets (either consistently too small or blowing past the max), the target parameters in §7 need revision.
- If HNSW query times exceed 100ms at classroom-tier corpus size, `ef_search` and possibly `m/ef_construction` need re-tuning.
- If thin-grounding events fire on more than 15% of retrievals at MVP scale, either the thresholds are wrong or the curated pointer sets are genuinely too thin — either way, the response depends on which.
- If pre-warming's cost line grows visible in `cost_ledger`, the warming schedule (§14) needs tightening.
- `chunk_relations` for parent-child chunk hierarchies (splitting oversized code and math blocks with sibling links) — deferred from §7 pending observed need.

19. Forward references and open questions

Items this spec deliberately punts to later specs or to build-time discovery.

Data layer spec (v1.2 pending). Two additions this subsystem needs: `source_chunks.chunk_type` (enum) and `source_chunks.tsvector_text` (generated column). Both are additive, spec'd in §5, and belong in the same v1.2 batch as the existing pending items.

Agent runtime spec (v1.1 pending). The Lecturer's `retrieve_passages` invocation pattern — how the agent constructs the query text, when it passes `None`, how it handles thin grounding — belongs in subsystem 2's spec. The current subsystem 2 v1.0 covers the invocation at a high level; v1.1 refinement based on this spec's contract is expected.

Ingestion spec (subsystem 5). Where the chunking algorithm from §7 is actually called. Where PDF text extraction produces the input to chunking. Where the `ingestion_jobs` for embedding are enqueued. Where `concept_sources` rows are authored (either by domain-expert workflow or by heuristic during ingestion). Where source licensing metadata is captured and where the human-in-the-loop review of new sources happens.

Frontend spec (subsystem 4). How citation markers render as hover cards. How the passage view expands on click. How thin-grounding warnings appear to the learner (if at all — the current design has them appear only in the reviewer’s queue).

Evaluation spec (subsystem 6). How retrieval quality is measured. Golden datasets of query-passage pairs against which retrieval regression tests run. Per-modality accuracy metrics (vector recall@10, keyword precision@10). This subsystem provides the hooks; the datasets and metrics live in subsystem 6.

Infrastructure spec (subsystem 7). Voyage API key management. Rate limit handling if Voyage’s per-org quota becomes binding. Alternate provider failover (Cohere Rerank 3, OpenAI embeddings) if Voyage has a sustained outage. Redis cache for multi-node retrieval-result caching.

Open questions requiring build-time answers.

1. **Real chunk-size distribution.** The 400-token target with 600-token max is based on rule-of-thumb chunking parameters, not measurement against Studium’s specific sources. Once real corpora are chunked, the distribution of actual chunk sizes needs a look; if a majority of chunks are hitting the max size, the target should probably be reduced.
 2. **Curated-vs-expanded ratio.** How often does retrieval return curated chunks versus vector-expanded chunks? A healthy MVP pattern is curated-dominant (curator authored the graph carefully); a pattern where >80% of results come from expansion means the concept-sources authoring is behind and needs catching up.
 3. **Reranker measurable improvement.** Voyage rerank-2 is chosen on the assumption that it materially improves ordering over raw hybrid-search fusion. This should be measured after MVP launch against a hand-labelled evaluation set (a subsystem 6 concern). If the improvement is marginal, reranking’s per-call cost may not justify the complexity.
 4. **Voyage-3 vs alternatives.** Voyage is chosen on quality/cost balance at spec time. Anthropic’s own future embedding offering, or improved Cohere/OpenAI options, may warrant reconsideration. The `EmbeddingProvider` protocol makes this a one-line change; the decision framework belongs in subsystem 7.
 5. **Cross-subject retrieval demand.** MVP restricts every call to a single subject. If learners studying two subjects consistently ask questions where the answer sits in the other subject’s corpus, the constraint may need to relax. Measurable from journal entries and session turns; a v2 concern.
-

End of specification

This document defines the retrieval subsystem for Studium in full. Chunking, embedding, hybrid search, reranking, concept-graph constraints, stance-based weighting, citation resolution, and thin-grounding detection are specified end-to-end. A senior engineer with data layer v1.1, agent runtime v1.0, and this document — plus a Voyage API key for Tier 3 tests — can build a working retrieval subsystem that agents can call, sources can be embedded into, and citations can be resolved through.

Next in sequence: **Subsystem 4 — Frontend and Interaction**, which specifies the Next.js application that consumes agent streams, renders lectures with hoverable citations resolved via §12's endpoint, exposes the eight tutorial primitives as UI affordances, handles the “raise your hand” interruption, and meets WCAG 2.2 Level AA. Written against data layer v1.1, agent runtime v1.0, and this document.